WEEK 2 / LECTURE 04

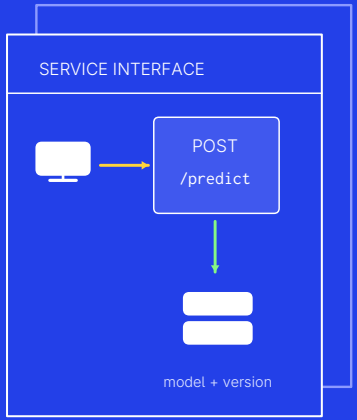
MLOps: Machine Learning Operations

API Implementation

From REST contracts to maintainable ML prediction services.

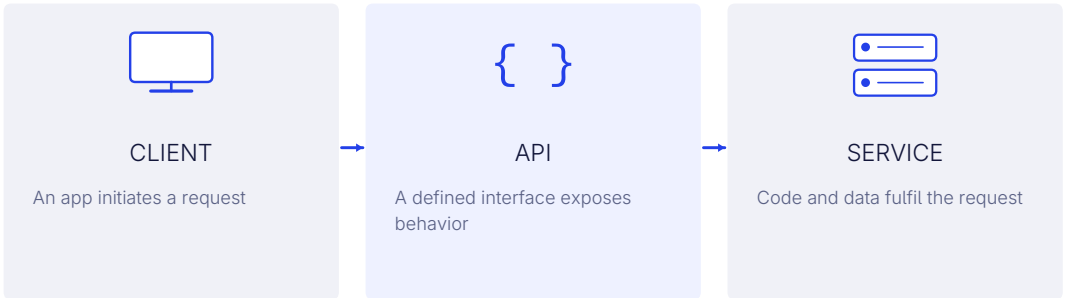
TRAN Cong Minh

CY Cergy Paris Université · ETIS / ENSEA



An API is a contract between software components

The client sends a request; the service returns a response.



A contract describes what can be requested and what can be returned.

APIs are already everywhere

One interface lets many kinds of software cooperate.



MEDIA

Search videos, metadata or channels via a documented API.



COMMERCE

Retrieve catalog details, prices, stock or an order status.



MLOPS

Request a trained-model prediction from another application.

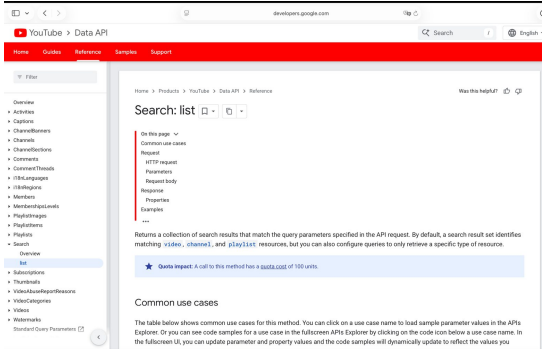
API != user interface: clients can be browser apps, scripts or other services.

Example · YouTube Data API

A real API reference makes its endpoints and parameters discoverable.

LOOK FOR

- Resource: search results
- Endpoint: search.list
- Parameters, quotas and response fields are documented



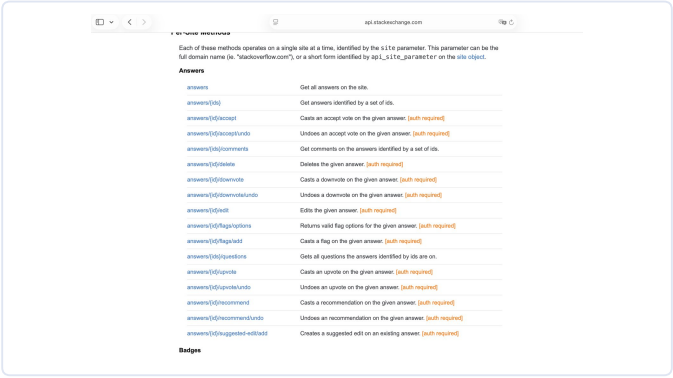
Screenshot from supplied lecture, p. 4 · Google API documentation

Example · Stack Exchange API

Endpoints describe operations on questions, answers and other resources.

LOOK FOR

- Examples: /answers and /answers/{ids}
- Certain operations require authentication
- Documentation acts as a contract for clients



Screenshot from supplied lecture, p. 5 · Stack Exchange API

Who can access the interface?

Availability and authorization are independent design decisions.



PRIVATE API

Access within one organization or an approved internal boundary.



PUBLIC API

Published for third-party integration, often with keys or quotas.



PARTNER API

Shared with approved collaborators under restricted access.

A public API need not be free or unauthenticated. Source: Postman.

API technologies belong to different categories

REST, RPC, SOAP and WebSocket are not four equivalent transport protocols.

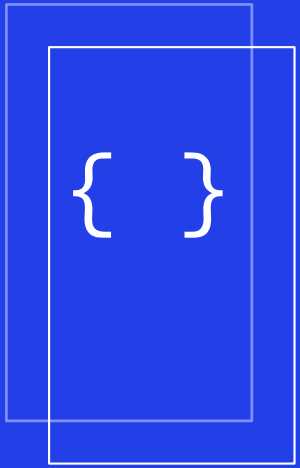
TECHNOLOGY	WHAT IT IS	PRACTICAL ROLE
REST	Architectural style (constraints)	Resource-oriented web APIs
RPC / gRPC	Interaction model / RPC framework	Remote operations and services
SOAP	XML messaging protocol	Contract-driven integrations
WebSocket	Two-way network protocol	Long-lived bidirectional messaging

These concepts operate at different layers; the taxonomy is explanatory, not exhaustive.

SECTION 01

REST & HTTP

Resources, methods, messages and status codes.



REST is an architectural style, not a file format

Fielding described constraints for distributed hypermedia systems.



JSON over HTTP is not, by itself, proof that an API follows REST.

GET · retrieve a resource representation

GET /users/123 · READ

REQUEST

```
GET /users/123 HTTP/1.1
Host: api.example.com
Accept: application/json
```



RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":123,"name":"Alice"}
```

GET requests a representation; it does not ask to change server state.

Restaurant analogy: asking to see the menu.

POST · ask the target to process data

POST /users · CREATE (EXAMPLE)

REQUEST

```
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Content-Length: 14

{"name":"Bob"}
```

RESPONSE

```
HTTP/1.1 201 Created
Location: /users/124
Content-Type: application/json

{"id":124,"name":"Bob"}
```

POST can create a resource, but has other processing semantics too.

Restaurant analogy: placing a new order.

PUT · create or replace a target resource

PUT /users/123 · REPLACE

REQUEST

```
PUT /users/123 HTTP/1.1
Host: api.example.com
Content-Type: application/json
Content-Length: 35

{"name":"Alice","email":"a@ex.com"}
```



RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":123,"name":"Alice",
 "email":"a@ex.com"}
```

PUT replaces the target state; if it creates a resource, return 201.

Restaurant analogy: replacing an entire order.

PATCH · apply a change document

PATCH /users/123 · PARTIAL UPDATE

REQUEST

```
PATCH /users/123 HTTP/1.1
Host: api.example.com
Content-Type: application/merge-patch+json
Content-Length: 22

{"email":"new@ex.com"}
```

RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":123,"email":"new@ex.com"}
```

PATCH semantics depend on the patch format; this uses JSON Merge Patch.

Restaurant analogy: changing one item in an order.

DELETE · request removal of a resource

DELETE /users/123 · REMOVE

REQUEST

```
DELETE /users/123 HTTP/1.1
Host: api.example.com
```



RESPONSE

```
HTTP/1.1 204 No Content
```

204 reports successful processing and must not include response content.

Restaurant analogy: cancelling an order.

HTTP methods are often used for CRUD operations

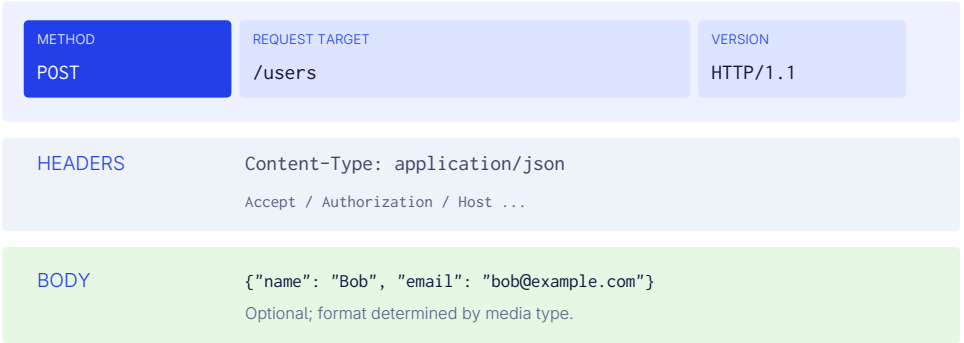
A useful design convention—not a formal definition of REST.

ACTION	COMMON METHOD	EXAMPLE
Create	POST	POST /users
Read	GET	GET /users/123
Create / Replace	PUT	PUT /users/123
Partial update	PATCH	PATCH /users/123
Delete	DELETE	DELETE /users/123

Semantics are standardized in RFC 9110 and RFC 5789; exact API operations vary.

Anatomy of an HTTP request

Method and target identify the action; headers describe the message.



HTTP/1.1 uses a start line, headers, a blank line, and an optional body.

A request is text with structure

Read the first line, headers and optional body separately.

ILLUSTRATIVE EXAMPLE

```
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: application/json
Content-Length: 49

{
  "name": "Bob",
  "email": "bob@example.com"
}
```

ANATOMY

METHOD + TARGET

Where the request goes.

HEADERS

How to interpret body.

BODY

Data submitted to the server.

Illustrative HTTP/1.1 example; bodies are optional for many requests.

ONE ENGINEERING IDEA

Identification and location are related, but not identical.

URI

or URL?

DISCUSS WITH A PARTNER

A relative path is not a complete URL

RFC 3986 distinguishes URIs from relative URI-references.



URI-REFERENCE

May be relative

/users/123 is a relative reference that needs a base URI to resolve.



ABSOLUTE URL

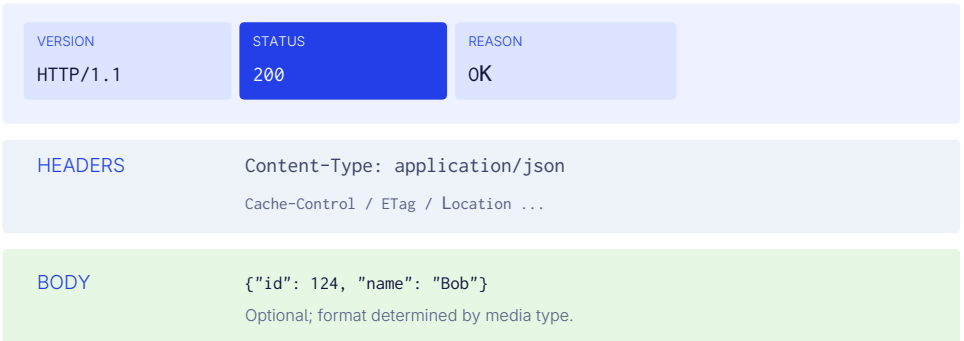
Scheme and authority

https://api.example.com/users/123 is an absolute HTTPS URI / URL.

Every URL is a URI; not every URI-reference is an absolute URI (RFC 3986).

Anatomy of an HTTP/1.1 response

First interpret the status; then the headers and optional content.



An HTTP/1.1 response uses a status line, header fields and optional body.

Read the response status before the JSON

Status, headers and body answer different questions.

ILLUSTRATIVE EXAMPLE

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{
  "id": 123,
  "name": "Alice"
}
```

RESPONSE

200 OK

Request handled successfully.

Content-Type

Interpret data as JSON.

JSON BODY

Representation returned.

The message body can be JSON, text, HTML, binary data or absent.

HTTP status codes convey five outcome classes

The leading digit tells you the broad category of result.

1xx	2xx	3xx	4xx	5xx
INFORMATIONAL	SUCCESS	REDIRECTION	CLIENT ERROR	SERVER ERROR
Continue process	Request completed	More action / redirect	Request cannot be fulfilled	Service failed
`100`	`200`	`301`	`404`	`503`

Use the full status and the response body to diagnose the specific outcome.

Status codes to recognize in API work

Return codes must match the actual outcome, not a generic error template.

CODE	MEANING	WHEN TO EXPECT IT
200 / 201	OK / Created	Successful operation / resource created
204	No Content	Success with no response body
301	Moved Permanently	Permanent redirect (typically navigation)
400 / 422	Bad Request / Unprocessable Content	Invalid syntax / semantic validation
401 / 403	Unauthorized / Forbidden	Authentication needed / access refused
404	Not Found	Target resource was not found
500 / 503	Server Error / Service Unavailable	Server fault / temporary unavailability

FastAPI returns 422 by default for request-model validation failures.

An endpoint is method + resource path

A small API often follows predictable collection/item patterns.

TARGET	GET	POST	PATCH / PUT	DELETE
/surveys	List surveys	Create survey	Usually not here	Design-dependent
/surveys/123	Get survey 123	Design-dependent	Update survey 123	Delete survey 123
/surveys/123/responses	List responses	Add response	Design-dependent	Design-dependent
/responses/42	Get response 42	Design-dependent	Update response 42	Delete response 42

Examples adapted from the original lecture; concrete behavior depends on the API specification.

HTTP operates inside a larger web architecture

The request may cross infrastructure before reaching application code.



FIREWALL

Allow or block traffic according to network or application rules.



ROUTER / PROXY

Choose a backend based on a path, hostname or policy.



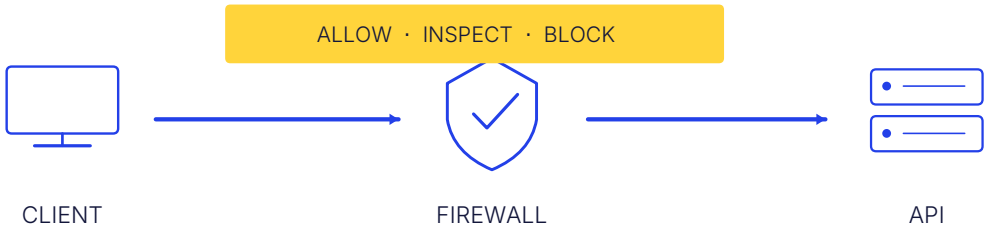
CACHE

Reuse eligible responses to reduce repeated work.

Real deployments may also include TLS termination, load balancing and observability.

A firewall enforces an access boundary

Network firewalls and web application firewalls inspect different information.



NETWORK RULES	IP, port, TLS, connection state
WEB APP FIREWALL	HTTP paths, headers and sometimes body

Correction: a WAF may inspect request bodies; filtering is not restricted to HTTP headers.

Useful conventions for a resource API

Consistency helps clients predict behavior; REST does not mandate path spelling.



RESOURCE NOUNS

GET /users instead of GET /get_users is a common convention.



CLEAR SCHEMAS

Document paths, query inputs, headers and response shapes.



USEFUL ERRORS

Use the correct HTTP status and an informative safe message.

Plural nouns and hyphens are conventions, not architectural requirements.

ONE ENGINEERING IDEA

The interface contract can also be machine-readable.

OpenAPI

& Swagger

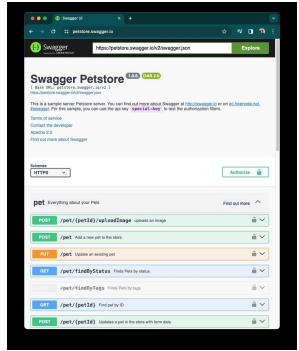
DISCUSS WITH A PARTNER

OpenAPI specifies contracts; Swagger provides tools

OAS is a standard description for HTTP APIs, not limited to REST.

LOOK FOR

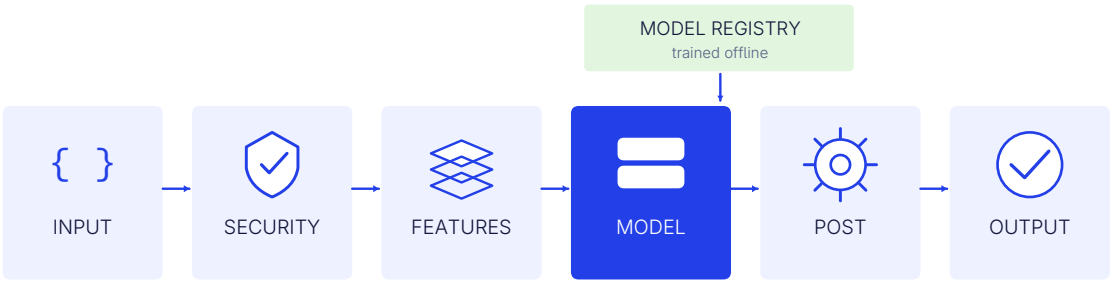
- OpenAPI: machine-readable operations and schemas
- Swagger UI: browsable, interactive API explorer
- Document parameters, responses and security



Source screenshot: original lecture p. 29, Swagger Petstore; see spec.openapis.org.

An ML API is more than model.predict()

The service boundary includes input validation, inference and response formatting.

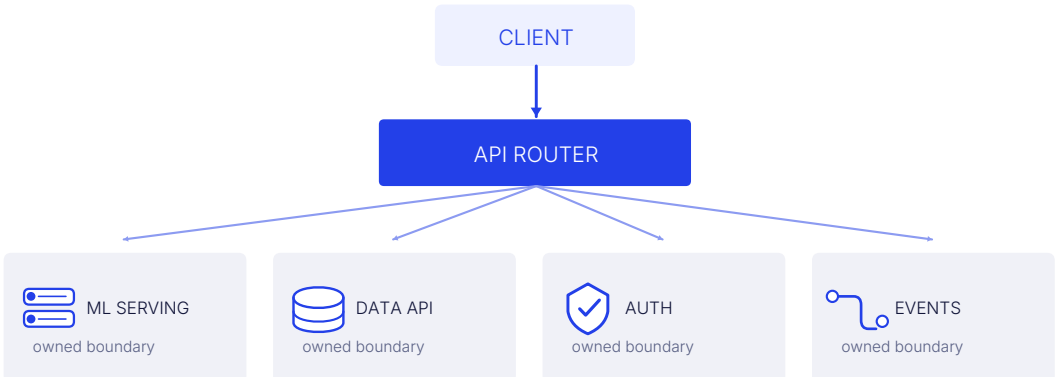


VALIDATE → INFER → RESPOND

Training and registering a model typically happen offline; serving loads a trusted artifact.

A product may expose several independent services

The API layer coordinates capabilities; not every operation belongs in one model service.

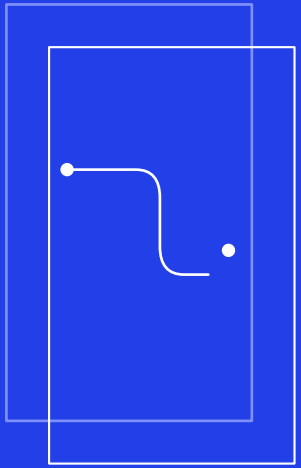


Separate services only when independent deployment and ownership justify the overhead.

SECTION 02

RPC & gRPC

Action-oriented alternatives to resource-oriented APIs.



RPC models a remote operation as a call

Clients request an operation with parameters; servers return a result.



REST

Resource-oriented

GET /users/123 asks for a resource representation.



RPC

Action-oriented

getUser({id: 123}) names the action to execute.

Neither style is automatically better; choose based on boundaries and client needs.

REST vs JSON-RPC · retrieving a user

Compare request and response separately in each protocol example.

REST / HTTP

REQUEST

```
GET /users/123 HTTP/1.1
Host: api.example.com
```

RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":123,"name":"Alice"}
```

JSON-RPC 2.0

REQUEST

```
{"jsonrpc":"2.0",
 "method":"getUser",
 "params":{"id":123},"id":1}
```

RESPONSE

```
{"jsonrpc":"2.0",
 "result":{"id":123,"name":"Alice"},
 "id":1}
```

JSON-RPC 2.0 envelopes carry method, params, id and result; both examples are illustrative.

REST vs JSON-RPC · changing an email

An HTTP method or a named operation can express a state change.

REST / HTTP

REQUEST

```
PATCH /users/123 HTTP/1.1
Host: api.example.com
Content-Type: application/merge-patch+json
Content-Length: 22

{"email":"new@ex.com"}
```

RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":123,"email":"new@ex.com"}
```

JSON-RPC 2.0

REQUEST

```
{"jsonrpc":"2.0",
 "method":"updateUserEmail",
 "params":{"id":123,"email":"new@ex.com"},
 "id":2}
```

RESPONSE

```
{"jsonrpc":"2.0",
 "result":{"success":true},"id":2}
```

This REST example uses JSON Merge Patch; the original lecture used a PUT example.

Remote invocation looks like a local call

But serialization, transport errors and latency cross a network boundary.

ILLUSTRATIVE EXAMPLE

```
# Illustrative client-side pseudocode
result = client.get_user(id=123)

# Encoded request sent over the network
# Server dispatches get_user(123)

# Structured result returned to client
print(result.name)
```

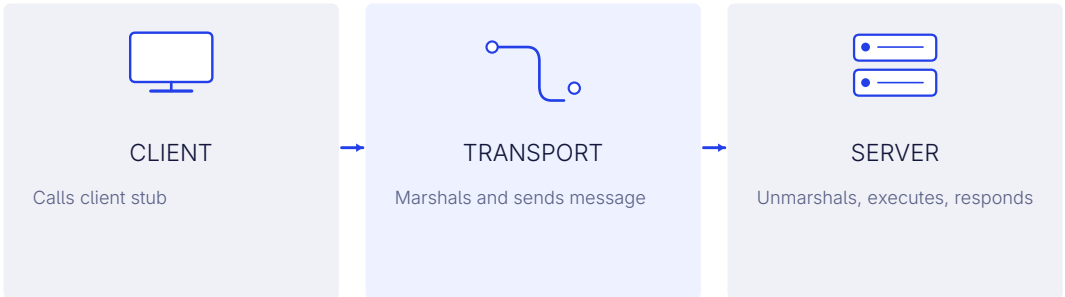
INVISIBLE WORK

- Serialize arguments
- Send the request
- Dispatch on server
- Return result or error

Illustrative pseudocode, not a runnable SDK example.

RPC runtime and stubs manage the boundary

Both client and server participate in encoding the call.



Remote calls can fail even when a local function call would not.

Stateless and stateful describe interaction context

REST statelessness concerns request context, not the absence of server data.



STATELESS REQUEST

Independent message

Each request carries what is needed to understand it; servers can still store resources.



STATEFUL SESSION

Context retained

A workflow may rely on session information from preceding interactions.

RPC can support both models; statefulness is not defined by the use of RPC alone.

Cookies can support either session architecture

The key question is where the server needs to keep interaction context.



COOKIE

Client sends the token



STATELESS CASE

Self-contained credentials can be verified



STATEFUL CASE

Session ID may require a server-side lookup

REST statelessness is about each request containing its required context.

Four building blocks of RPC systems

A call is a protocol exchange, not merely a Python method.



INVOCATION

Named operations and parameters define the API.



STUBS

Client/server components serialize and parse messages.



IDL / TYPES

A shared interface describes methods and message shapes.

Failure handling and transport remain part of the design, even with typed stubs.

gRPC uses typed contracts and HTTP/2

A mature RPC framework originally developed at Google.



PROTOBUF

Protocol Buffers commonly define service methods and message types.



HTTP/2

Efficient framing and multiple concurrent streams over one connection.



STREAMING

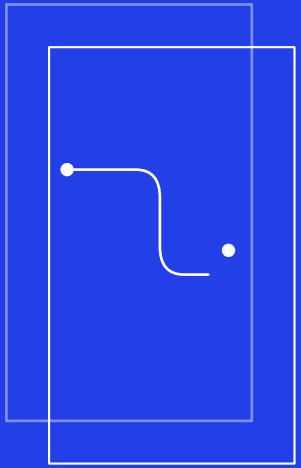
Supports unary calls and client/server/bidirectional streaming.

Authentication and deadline handling are important application concerns.

SECTION 03

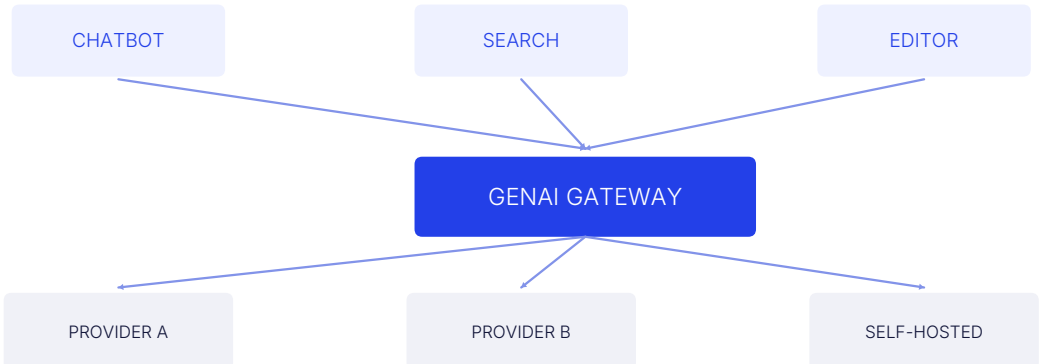
GenAI API Gateways

One managed access layer for multiple foundation-model providers.



A GenAI gateway centralizes model access

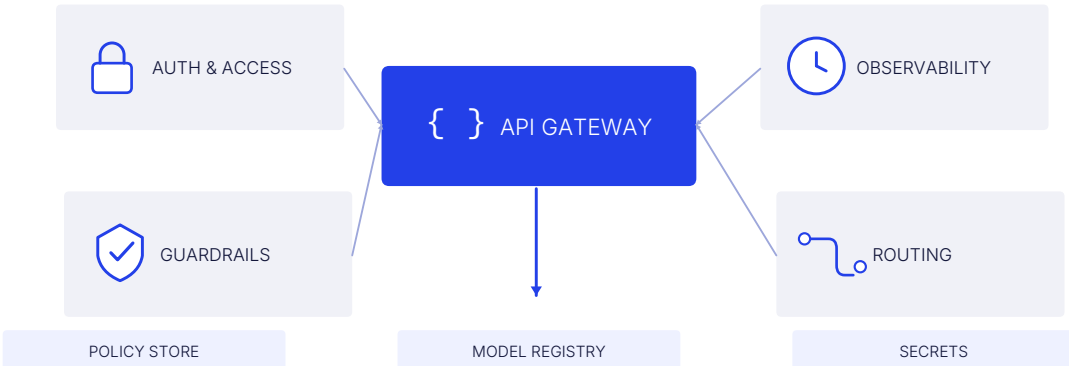
Applications call one managed API rather than integrating every provider.



A gateway can centralize routing, credentials, policies and usage telemetry.

What belongs inside a GenAI gateway?

Separate runtime request handling from supporting governance stores.



A practical architecture can evolve from a small gateway to these capabilities.

Why organizations introduce model gateways

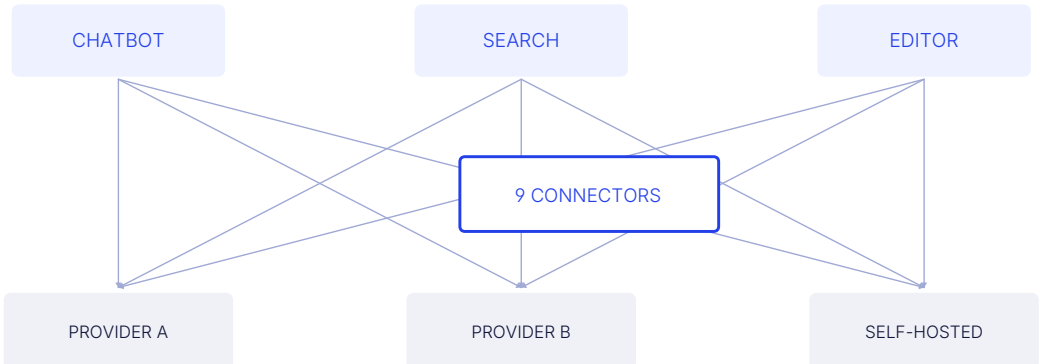
Centralizing common concerns reduces duplicated integration work.

CAPABILITY	VALUE
Model selection	Control available providers and compare cost, quality and policy.
Shared tooling	Reuse caching, metrics, retries and credentials.
Cost and quotas	Attribute spend to applications; apply budgets and limits.
Sensitive-data controls	Enforce approved providers and data-handling policies.
Guardrails	Apply checks and record residual risks across clients.

Guardrails mitigate risks; they cannot guarantee factuality or absence of harm.

Without a gateway, every client integrates every model

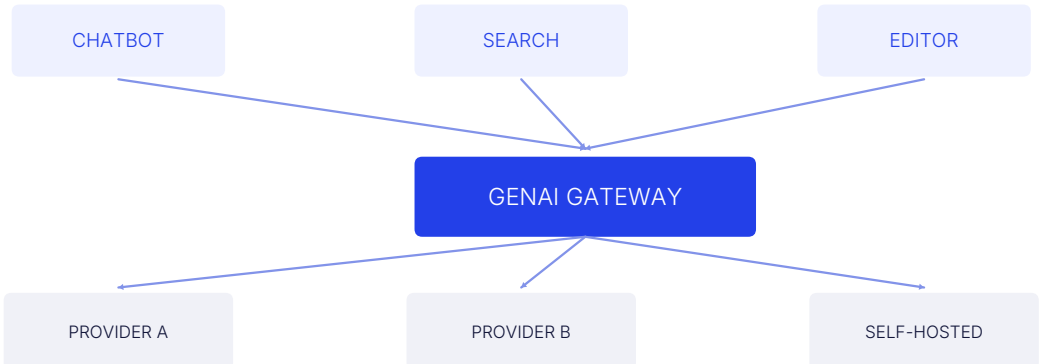
N × M connections multiply integration and governance effort.



Direct provider integration can still make sense for small, specialized systems.

With a gateway, clients share one managed boundary

Providers remain heterogeneous behind a unified organization-level contract.



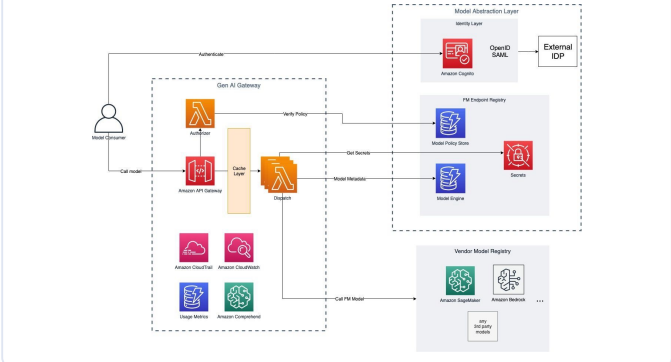
The gateway itself becomes infrastructure that requires reliability and monitoring.

Reference architecture: a managed GenAI entry point

The original lecture uses an AWS architecture as one possible implementation.

LOOK FOR

- Model abstraction layer: registry, identity and policy
- Gateway: request forwarding, limits, checks
- Supporting services: telemetry and secrets



Reference figure from AWS blog, reproduced in supplied lecture p. 48

The model abstraction layer governs choices

The consuming application should not manage every provider credential.



REGISTRY

Available models, endpoints and model metadata.



POLICY + IDENTITY

Model access rights, locations, allowed use and quotas.



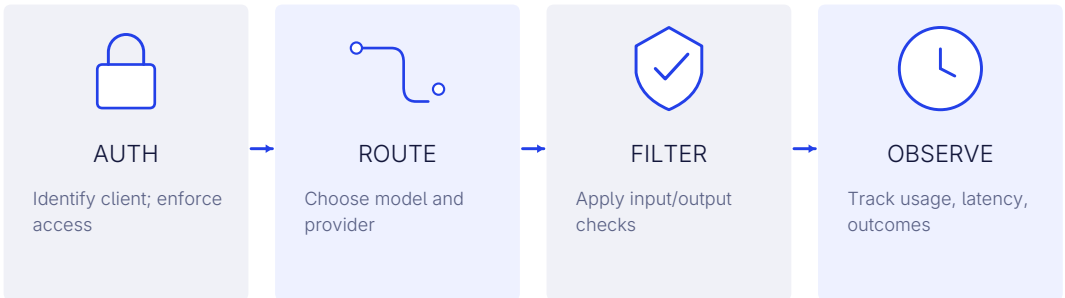
SECRETS

Secure storage and rotation of provider credentials.

Reference architecture: AWS generative AI gateway blog (as in source lecture).

A gateway processes more than the request payload

One call can trigger several policy and runtime checks.



Consistent contracts simplify clients; policies must remain observable.

Control cost, traffic and incident evidence

Operational controls make shared GenAI infrastructure manageable.

CONTROL	WHAT IT DOES	WATCH OUT FOR
Cache	Reuse responses where safe	Sensitive or personalized outputs
Rate limits	Bound request frequency	429 responses and retry behavior
Quotas	Allocate model usage budgets	Budget exhaustion and attribution
Observability	Metrics, tracing, safe logs	Never log secrets or unnecessary personal data
Audit trails	Document who called what	Retention and access policies

Cacheability depends on content and provider agreements, not just API method.

A minimal model registry and invocation interface

Conceptual Python-style contract from the GenAI gateway example.

ILLUSTRATIVE EXAMPLE

```
# Illustration — not a tested implementation
class ModelGateway:
    def list_models(self) -> list[dict]:
        ...

    def invoke(self, model_id: str,
               model_config: dict) -> dict:
        ...
```

PROVIDER BOUNDARY

List only approved models.

Validate permissions and input.

Route the invocation.

Record safe usage metadata.

Adapted conceptually from the supplied AWS gateway examples, pp. 52–53.

Provider-native API shapes are heterogeneous

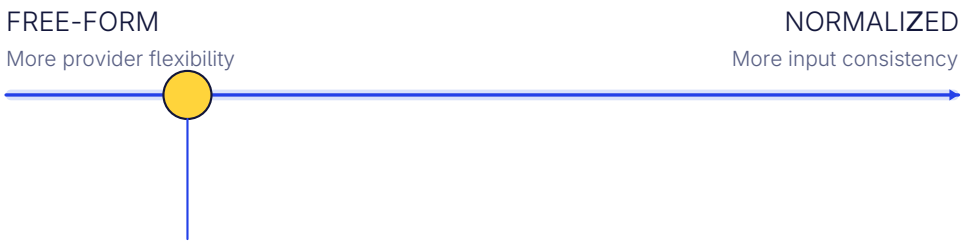
A gateway adapter must handle version and provider differences.

FAMILY	OPERATIONS IN THE SOURCE	WHAT TO VERIFY
OpenAI API	Chat Completions; Embeddings	Current version and available methods
Amazon Bedrock	InvokeModel; Converse	Model compatibility and API schema
Google Vertex AI	Predict model operation	Endpoint version and request shape

Names are examples drawn from source p. 53, not a guaranteed current endpoint contract.

Endpoint design is a trade-off

More flexibility transfers responsibility to the client.



Choose abstraction depth based on supported models and the validation boundary.

General invoke · a flexible gateway contract

A proposed gateway lets the client supply model-specific options.

ILLUSTRATIVE EXAMPLE

```
POST /models/{model_id}/invoke
Content-Type: application/json

{
  "input": ["Hello, world!"],
  "model_config": {
    "truncate": "START"
  }
}
```

TRADE-OFF

+ Flexible model options
+ Thin routing layer
- Harder to validate
- Caller knows provider details

HYPOTHETICAL interface, adapted from source p. 55; not a public provider schema.

A normalized conversation endpoint is more opinionated

Clearer shared fields, but less direct exposure of provider-specific features.



Provider adapters and contract evolution create maintenance work.

Converse · a constrained gateway contract

Shared expected fields simplify validation across supported backends.

ILLUSTRATIVE EXAMPLE

```
POST /models/{model_id}/converse
Content-Type: application/json

{
  "messages": [
    {"role":"user", "content":"Hello!"}
  ]
}
```

TRADE-OFF

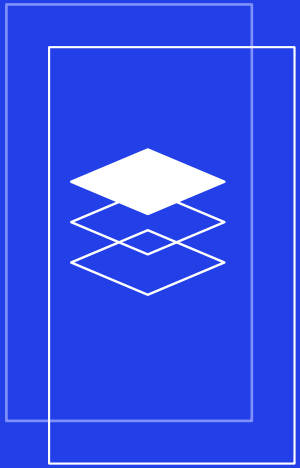
+ Uniform schema
+ Easier validation
- Not every feature exposed
- Adapter maintenance

HYPOTHETICAL gateway endpoint; not the provider-native Bedrock Converse schema.

SECTION 04

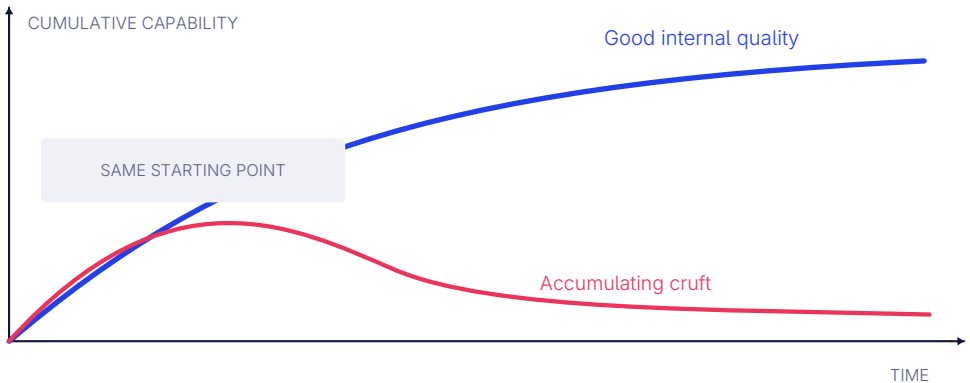
Microservice architecture

Better boundaries make change safer and cheaper.



Architecture determines the cost of future change

Good internal structure delays the growth of accidental complexity.

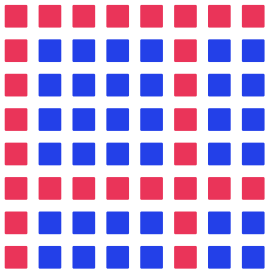


Conceptual illustration; curves are qualitative, not measured data. Adapted from source p. 59.

Cruft makes each new feature harder

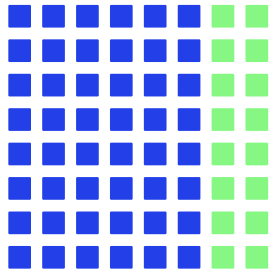
Duplicated logic, outdated abstractions and hidden dependencies accumulate.

ENTANGLED SYSTEM



Shared change creates collateral risk

CLEAR BOUNDARIES

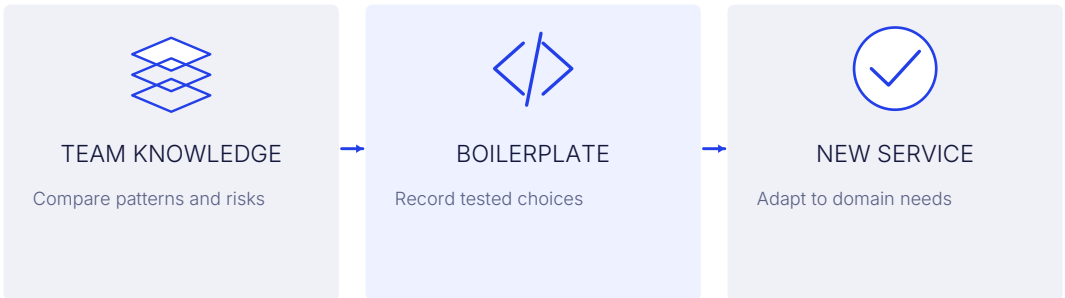


Changes stay within modules

Cleaning design debt is most valuable when it removes repeated future friction.

A boilerplate codifies team design decisions

Capture common architecture so teams do not rediscover it for each service.



A starting template is a guide, not a mandate to adopt every component.

What a good service template gives a team

Shared defaults are most useful when easy to understand and change.



FAST START

Reduce repetitive setup for each new repository.



CONSISTENCY

Use familiar layout, error contracts and test conventions.



EVOLUTION

Replace adapters without rewriting core business rules.

Templates need ongoing ownership; frozen boilerplates become cruft.

ONE ENGINEERING IDEA

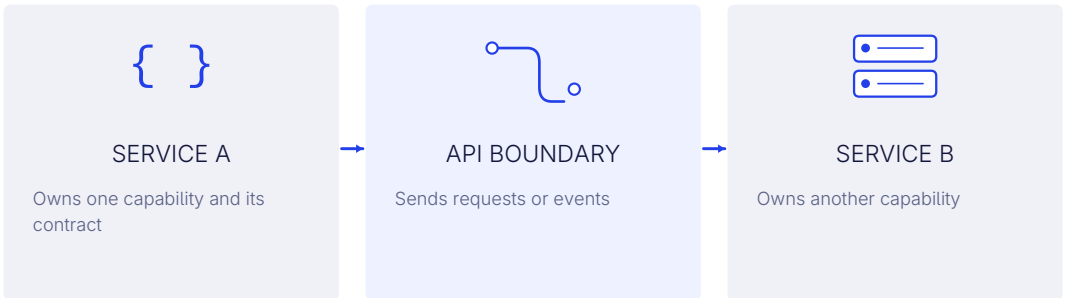
An independently deployable unit organized around one cohesive capability.

What makes a microservice?

DISCUSS WITH A PARTNER

A microservice owns a narrow business capability

Services communicate through explicit APIs and can be independently deployed.



Microservices are a design choice, not a default measure of system quality.

Microservice properties worth remembering

Small enough to understand, large enough to own meaningful behavior.



ONE RESPONSIBILITY

A cohesive domain capability and clear ownership.



INDEPENDENT CHANGE

Deploy, version and scale service boundaries when needed.



EXPLICIT CONTRACT

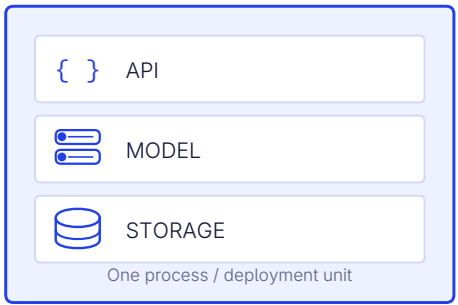
Stable interfaces; implementation details stay behind them.

Trade-offs: network failures, distributed data and operational overhead.

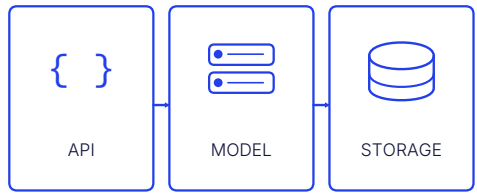
Monolith vs microservices · the real trade-off

A modular monolith is often the simpler and safer starting architecture.

MODULAR MONOLITH



INDEPENDENT SERVICES



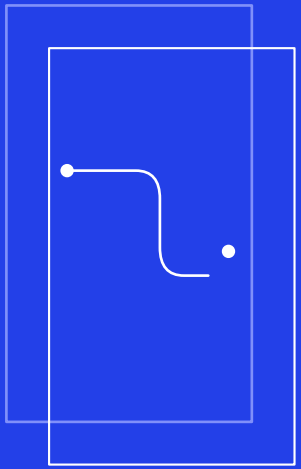
Network and deployment overhead

Choose microservices when team boundaries and independent delivery justify distribution.

SECTION 05

Abstractions & boundaries

Hide incidental complexity while keeping responsibilities explicit.



Abstraction exposes what matters now

An interface hides internal details without removing the underlying work.



PUBLIC VIEW

What the component offers

predict(features) → prediction



IMPLEMENTATION

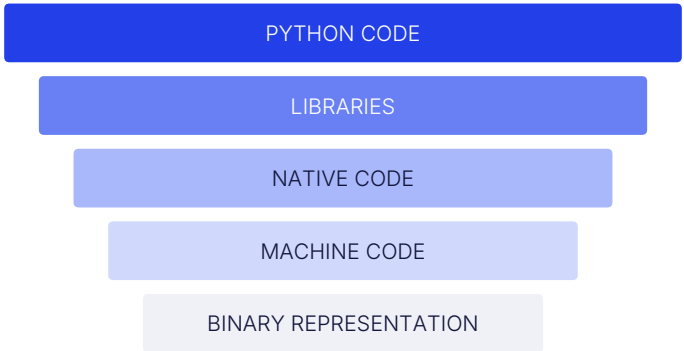
How the result is produced

Validation · preprocessing · model · response

The abstraction is useful only if its contract stays precise.

Abstractions build upon one another

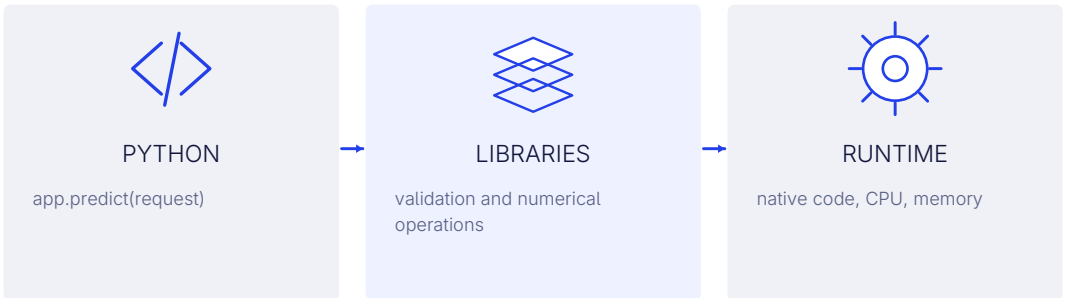
A higher-level program depends on lower-level mechanisms without reimplementing them.



Layers shown conceptually: Python → native code → instructions → machine state.

Python hides low-level mechanics behind readable APIs

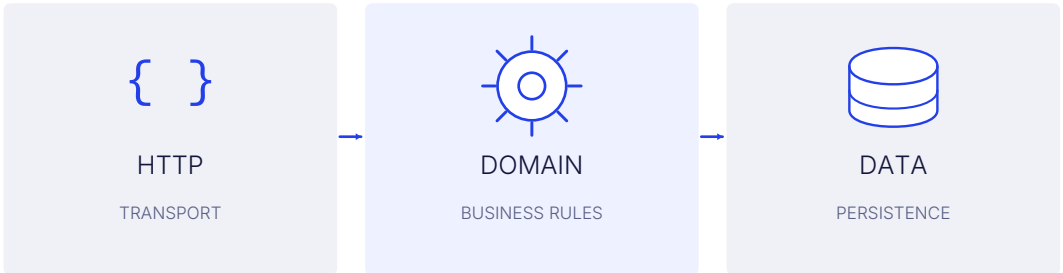
Software engineers work with abstractions at the level of their problem.



Abstraction reduces cognitive load, but runtime constraints still matter.

Separation of concerns gives each layer one job

Code becomes easier to reason about when changes have a clear home.



Examples: HTTP validation, business rules and database access should not be tangled.

Interfaces define the permitted interaction

Clients should depend on behavior, not implementation internals.



OPERATIONS

What actions are supported?



INPUTS / OUTPUTS

Which types, constraints and errors exist?



REPLACEABILITY

Can one implementation satisfy the same contract?

An identical method name alone does not guarantee semantic compatibility.

Interfaces can be explicit or structural

Python Protocols are optional static contracts; runtime behavior still matters.

JAVA / EXPLICIT

```
// Java-style illustration
interface Pet {
  String owner();
  void pat();
}
class Dog implements Pet {
  /* methods implemented here */
}
```

PYTHON / STRUCTURAL

```
# Python 3 typed illustration
from typing import Protocol
class Pet(Protocol):
    def owner(self) -> str: ...
    def pat(self) -> None: ...

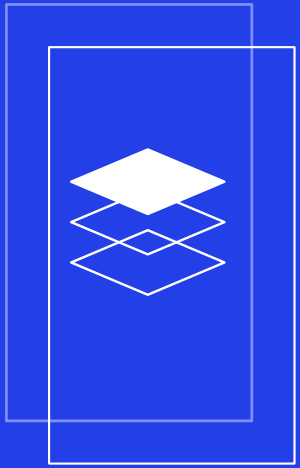
# Implementation is not shown
```

Both snippets are explanatory fragments rather than standalone runnable programs.

SECTION 06

Clean architecture

Core domain logic should not depend on external delivery technologies.



The dependency rule points toward the core

Outer technologies depend on inner policies; the core does not import their details.

DEPENDENCIES

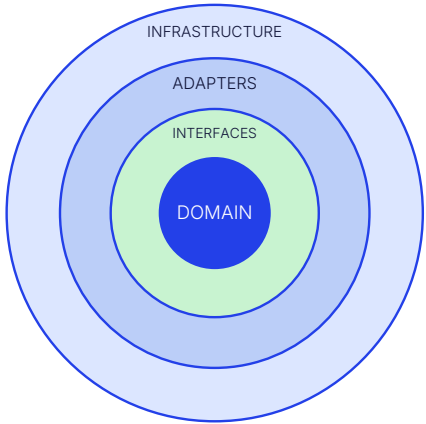
Point from outer layers toward the core.

INNER RULES

No framework or database imports in the core.

ADAPTERS

Translate external protocols and representations.



Based on Robert C. Martin's Clean Architecture; simplified adaptation, not an exact official figure.

Tailoring the layers to a Python service

Infrastructure supplies HTTP, storage and runtime; adapters isolate the domain.

INFRASTRUCTURE

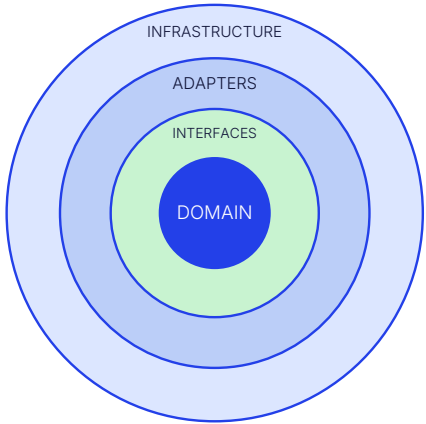
Web, database, runtime, environment.

INTERFACE ADAPTERS

HTTP handlers and IO gateways.

DOMAIN

Service-specific rules and invariants.

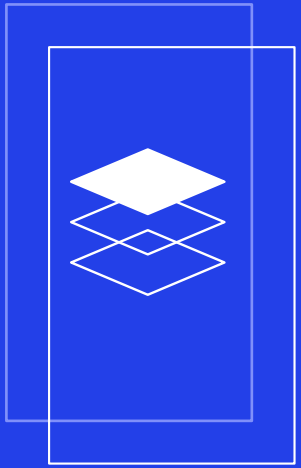


Framework, network and database choices should remain outside core business rules.

SECTION 07

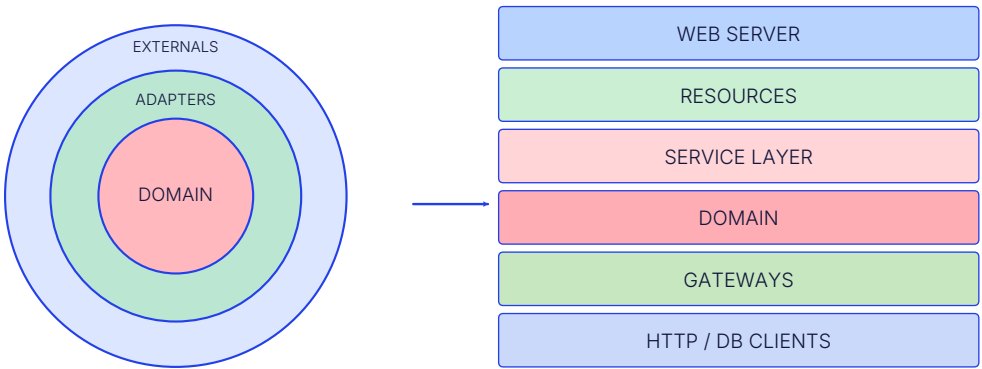
Anatomy of a microservice

A practical layered service structure, from transport to domain.



Map the components of a clean service

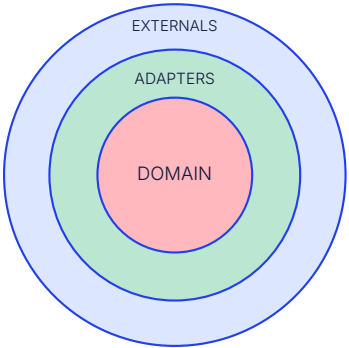
Transport and persistence are adapters around a core domain.

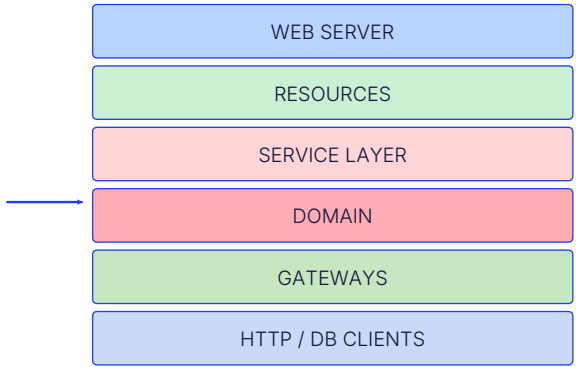


Redrawn from the microservice anatomy illustrated in source p. 78.

A minimal service template shares stable boundaries

Not every microservice needs every layer from day one.



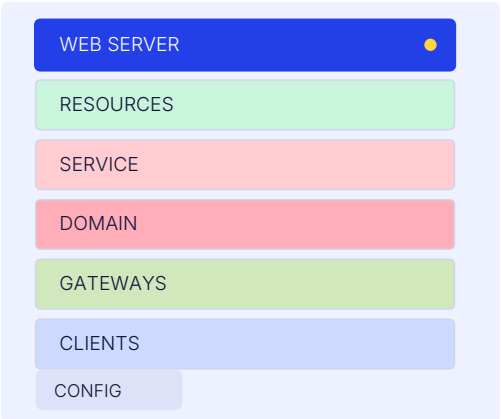


Keep only necessary components; preserve the boundary directions.

Web server · the protocol entry point

The framework receives HTTP and translates it into Python-facing request handling.

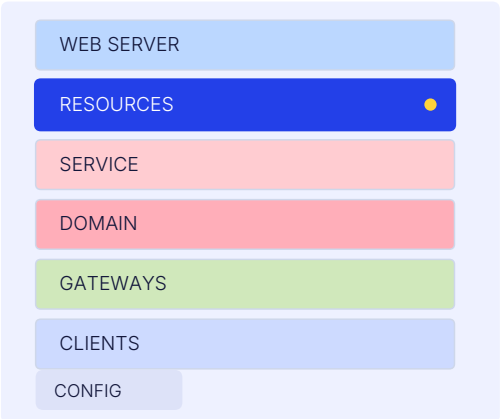
1. Manage the network protocol and endpoint dispatch.
2. Integrate request and response validation.
3. Keep business policy out of framework glue.



Resources · map transport to the domain

A thin adapter translates HTTP-specific request data into service operations.

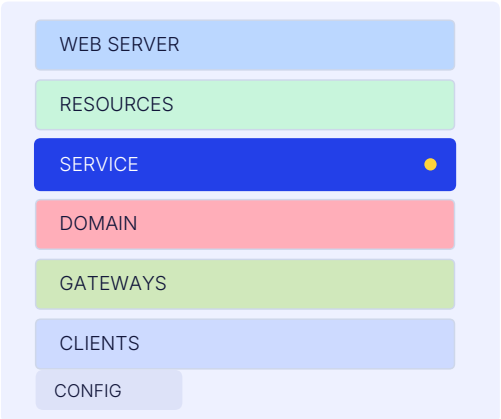
1. Validate and normalize incoming resource inputs.
2. Choose a domain operation to invoke.
3. Map service results and failures to HTTP output.



Service layer · expose use cases

Define reusable application operations independently of how clients reach them.

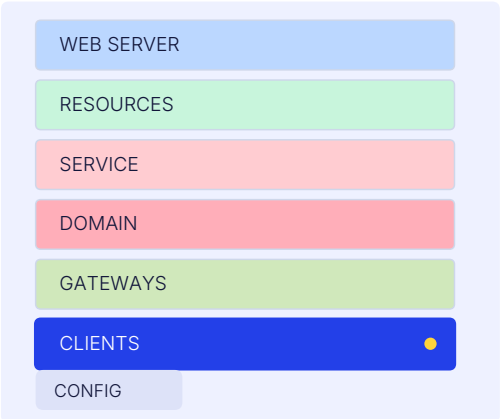
1. Expose cohesive operations, such as get_pet or add_pet.
2. Orchestrate domain operations and relevant gateways.
3. Remain callable from HTTP, messaging or scheduled jobs.



HTTP / DB clients · handle external protocols

A client adapter knows how to communicate with another system.

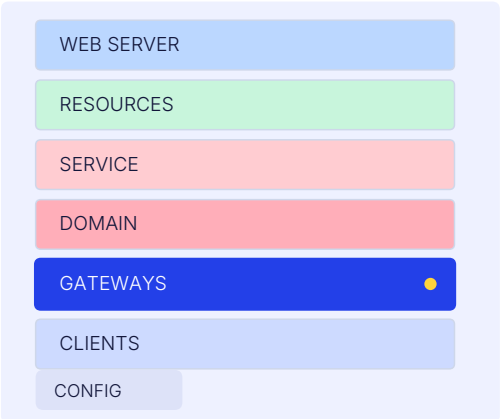
1. HTTP client: request/response, timeouts and errors.
2. DB client: database drivers and connection details.
3. Keep connection mechanics out of the domain.



Gateways · isolate outside systems

Encapsulate persistence and service integrations behind domain-friendly operations.

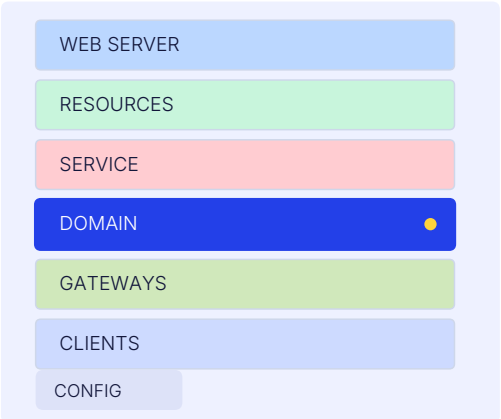
1. Keep queries and serialization in one place.
2. Translate external representations to internal values.
3. Make adapters replaceable in tests or future migrations.



Domain · the core meaning of the service

Business logic belongs in the layer with the fewest technical dependencies.

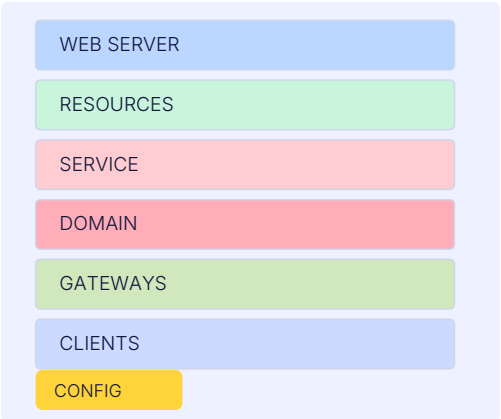
1. Define domain rules and invariants.
2. Depend on interfaces, not concrete databases or frameworks.
3. Avoid fitting or retraining the model during a prediction call.



Configuration · make runtime choices explicit

The same code can serve different environments via validated configuration.

1. Centralize paths, hostnames, timeouts and safe feature flags.
2. Read secrets from an approved secret mechanism.
3. Reject invalid combinations instead of silently defaulting.



Why this internal architecture can pay off

Clear ownership makes maintenance safer and concurrent work easier.



REPLACE EXTERNALS

Swap database driver or web framework behind adapters.



MAINTAIN

Develop modules without accidental cross-layer coupling.



DEPLOY

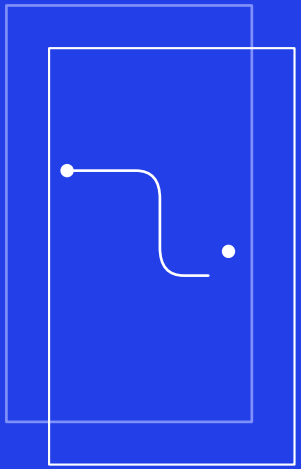
Configure the same reviewed code for different environments.

These benefits require discipline; adding unnecessary layers can make small systems worse.

SECTION 08

Worked example: Pets service

Trace a real service tree into its architectural responsibilities.



The pets example introduces a layered codebase

The screenshot shows selected packages, infrastructure files and an architecture map.

PETS SERVICE / SELECTED FILES

```
pets/
  app/flask/
    __init__.py
    routes.py
    error_handler.py
    core.py
  app/io/pets.py
  exceptions.py
  configs/
  config.py
  Dockerfile
  run.py
```

ARCHITECTURAL MAP

- app/flask: Flask and Connexion boundary
- app/io: persistence adapter code
- exceptions.py and config.py: shared support
- Dockerfile / run.py: runtime packaging

Source: original lecture p. 89. A partial tree is shown; repository not supplied.

Startup wires a Connexion / Flask app

The original example configures a specification-backed API at initialization.

SOURCE EXCERPT · UNTESTED

```
# SOURCE EXCERPT · app/flask/__init__.py
app = connexion.App(
    __name__,
    specification_dir=config.SPECIFICATION_DIR,
    options=options,
)
app.add_api("swagger.yaml",
    strict_validation=True,
    validate_responses=True)
```

OBSERVE IN SOURCE

Connexion is the web adapter.

OpenAPI file configures routes.

Validation happens at the edge.

Legacy sample, not executed.

Simplified from the original p. 90 screenshot; Connexion APIs differ by version.

The source labels HTTP-facing code as resources

The highlighted app/flask package connects external requests to internal operations.

PETS SERVICE / SELECTED FILES

```
pets/
  app/flask/
    __init__.py
    routes.py
    error_handler.py
    core.py
  app/io/pets.py
  exceptions.py
  configs/
  config.py
  Dockerfile
  run.py
```

RESOURCE ADAPTER

- The architecture marks Resources as an adapter.
- Endpoints and validation live near HTTP.
- No business rule should depend on HTTP types.

Source: original p. 91 highlights the Resources boundary; mapping is conceptual.

Core functions delegate to the IO package

The original service layer exposes named operations over the pets gateway.

SOURCE EXCERPT · UNTESTED

```
# SOURCE EXCERPT · app/flask/core.py
from app.io import pets

def get_all_pets():
    return pets.get_all()

def get_pet_by_id(pet_id):
    return pets.get(pet_id)
```

TRACE THE CALL

The HTTP handler calls core.

Core calls app.io.pets.

Business use case is distinct

from the storage adapter.

Source: original lecture p. 92; an excerpt of the shown functions.

core.py is mapped to the service layer

This is the explicit classification in the original progressive example.

PETS SERVICE / SELECTED FILES

```
pets/
  app/flask/
    __init__.py
    routes.py
    error_handler.py
    core.py
  app/io/pets.py
  exceptions.py
  configs/
  config.py
  Dockerfile
  run.py
```

SERVICE / USE CASE

- Service operations implement application use cases.
- Here they delegate to the gateway functions.
- A filename alone does not prove clean separation.

Source: original lecture p. 93 highlights core.py as Service Layer.

Custom errors are translated into HTTP

A resource-side handler maps an application exception to an HTTP response.

SOURCE EXCERPT · UNTESTED

```python
# SOURCE EXCERPT · error_handler.py
from flask import jsonify

def handle_custom_error(error):
    response = jsonify({"message": error.message})
    response.status_code = error.status_code
    return response
```

SOURCE MISMATCH

Handler uses error.message.

It also uses error.status_code.

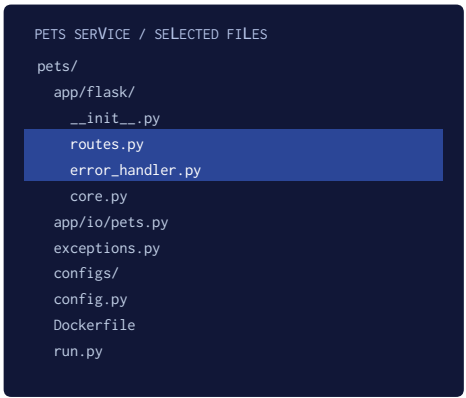
PetNotFound on p. 100

defines neither attribute.

Original pp. 94 & 100: the visible exception contract does not support this handler as shown.

Error handlers belong near the HTTP boundary

The original file mapping marks error_handler.py as a Resources concern.



RESOURCE ADAPTER

- Convert known failures into stable HTTP status.
- Document failure payloads in OpenAPI.
- Keep stack traces outside public responses.

Source: original p. 95; advice extends the screenshot, not a tested claim.

Routes delegate GET and DELETE to the core

The actual p. 96 screenshot shows routes.py, not the persistence adapter.

SOURCE EXCERPT · UNTESTED

```
# SOURCE EXCERPT · app/flask/routes.py
from app.flask import core

class Pet:
    @staticmethod
    def get(pet_id):
        return core.get_pet_by_id(pet_id), 200

    @staticmethod
    def delete(pet_id):
        return core.delete_pet_by_id(pet_id), 204
```

IMPORTANT

The route delegates to core.

204 MUST return no body.

Check framework serialization.

Legacy sample, not executed.

Source: original p. 96. The exact framework handling of the 204 body is unverified.

routes.py is an HTTP resource adapter

The highlighted source file connects public operations to core use cases.

PETS SERVICE / SELECTED FILES

```
pets/
  app/flask/
    __init__.py
    routes.py
    error_handler.py
    core.py
  app/io/pets.py
  exceptions.py
  configs/
  config.py
  Dockerfile
  run.py
```

RESOURCE ADAPTER

- HTTP verb and path map to a route handler.
- Handler calls a function in core.py.
- Returned status and body must follow HTTP semantics.

Source: original lecture p. 97 highlights routes.py as Resources.

The persistence adapter uses Firestore

The original app/io/pets.py screenshot performs external reads and raises a domain-specific error.

SOURCE EXCERPT · UNTESTED

```
# SOURCE EXCERPT · app/io/pets.py
client = firestore.Client()
pets = client.collection("pets")

def get(pet_id: str) -> dict:
    doc_ref = pets.document(pet_id)
    snapshot = doc_ref.get()
    if not snapshot.exists:
        raise exceptions.PetNotFound(...)
    return snapshot.to_dict()
```

SOURCE CHECK

Firestore is a concrete client.

Not-found raises PetNotFound.

The original literal error message omits an f-string prefix.

Source: p. 98; omitted error text contains unexpanded {pet_id} in the original screenshot.

app/io/pets.py is classified as a gateway

The source places external data access behind a reusable adapter.

PETS SERVICE / SELECTED FILES

```
pets/
  app/flask/
    __init__.py
    routes.py
    error_handler.py
    core.py
  app/io/pets.py
  exceptions.py
  configs/
  config.py
  Dockerfile
  run.py
```

GATEWAY / IO

- Gateway names expose use-case-friendly operations.
- Firestore SDK details stay in app/io.
- Service calls the adapter rather than the driver.

Source: original p. 99 highlights app/io/pets.py as Gateways.

PetNotFound defines an expected failure type

An explicit exception lets outer layers distinguish absence from a software defect.

SOURCE EXCERPT · UNTESTED

```
# SOURCE EXCERPT · exceptions.py
class PetNotFound(Exception):
    """Exception to raise when a pet
    is not found."""
```

SOURCE MISMATCH

Only Exception + docstring shown.

No .message / .status_code.

The handler on p. 94 may fail.

Define and test an error contract.

Source: original p. 100; available excerpt does not define attributes read by error_handler.py.

The source maps exceptions.py to service concerns

An exception type lets the service report expected problems without writing HTTP status codes.

PETS SERVICE / SELECTED FILES

```
pets/
  app/flask/
    __init__.py
    routes.py
    error_handler.py
    core.py
  app/io/pets.py
  exceptions.py
  configs/
  config.py
  Dockerfile
  run.py
```

SERVICE / USE CASE

- A named failure crosses the boundary.
- The HTTP handler chooses its external representation.
- This mapping is one example, not a universal rule.

Source: original p. 101 highlights exceptions.py as Service Layer.

Configuration resolves the specification location

The source computes the directory containing its API YAML files.

SOURCE EXCERPT · UNTESTED

```
# SOURCE EXCERPT · config.py
import os

SPECIFICATION_DIR = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "configs",
)
```

WHAT IT SHOWS

Filesystem path derived from file.

The YAML contract lives in configs/.

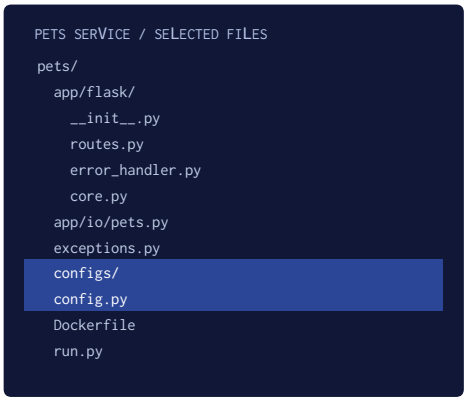
Configuration is separate

from domain behavior.

Simplified but semantically equivalent to original p. 102; no filesystem was tested.

config.py belongs to the runtime configuration

The original example highlights the environment and configuration boundary.



CONFIGURATION

- config.py defines a contract directory.
- Other runtime settings can live nearby.
- Validate deployment settings before serving.

Source: original lecture p. 103 explicitly highlights config.py as Config.

Match responsibilities to the shown source files

The original architectural diagram maps each file to a role.

PETS SERVICE / SELECTED FILES

```
pets/
  app/flask/
    __init__.py
    routes.py
    error_handler.py
    core.py
  app/io/pets.py
  exceptions.py
  configs/
  config.py
  Dockerfile
  run.py
```

COMPLETE LAYER MAP

- Connexion / Flask → web framework
- routes.py and error_handler.py → resources
- core.py and exceptions.py → service
- app/io/pets.py → gateway
- Firestore SDK → database client

Source: original p. 104; exact full repository content is unavailable.

Ask what belongs in the domain

The example explicitly raises the question of where domain logic should live.

PETS SERVICE / SELECTED FILES

```
pets/
  app/flask/
    __init__.py
    routes.py
    error_handler.py
    core.py
  app/io/pets.py
  exceptions.py
  configs/
  config.py
  Dockerfile
  run.py
```

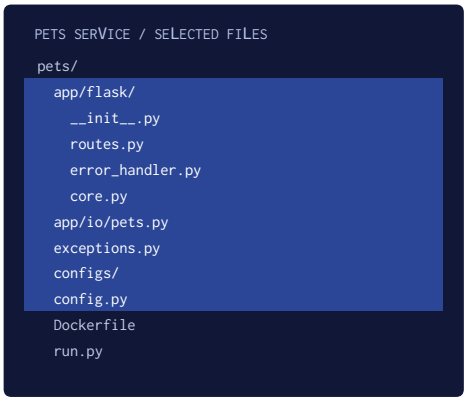
DOMAIN LOGIC

- A module name is not an architecture guarantee.
- Check imports and actual responsibilities.
- Move reusable rules away from transport and persistence.

Source p. 105 contains a question marker on the Domain layer.

Final source-to-architecture mapping

The last source diagram maps web, persistence, service and runtime dependencies.



COMPLETE LAYER MAP

- Web: Connexion / Flask
- Resources: routes.py, error_handler.py
- Service: core.py, exceptions.py
- Gateway: app/io/pets.py
- Clients: Requests / Google Cloud libraries
- Config: config.py; Dockerfile and run.py run it

Source: original p. 106; map derived from screenshot, not a repository audit.

ONE ENGINEERING IDEA

A documented interface, explicit errors and maintainable dependencies make ML serving operational.

That is the

API service boundary.

DISCUSS WITH A PARTNER

Primary specifications behind the API examples

Technical claims in this lecture are checked against named standards.

TOPIC	AUTHORITATIVE REFERENCE	SUPPORTED CONCEPT
REST	R. Fielding, PhD thesis (2000), Chapter 5	REST architectural constraints
HTTP semantics	IETF RFC 9110	Methods, status codes and responses
PATCH / JSON Merge Patch	IETF RFC 5789; RFC 7396	Partial updates and patch payloads
URI references	IETF RFC 3986	Relative path versus absolute URI
JSON-RPC	JSON-RPC 2.0 Specification	Request / response envelopes
OpenAPI	OpenAPI Specification 3.x	Machine-readable HTTP API descriptions

Sources: rfc-editor.org · fielding dissertation · jsonrpc.org · spec.openapis.org.

Implementation references and evidence limits

Distinguish documented examples from executed code and verified systems.

AREA	SOURCE	INTERPRETATION
gRPC	grpc.io/docs/what-is-grpc/	Transport, message types and streaming
FastAPI	fastapi.tiangolo.com	Default 422 validation; lifespan-based tests
GenAI gateway	AWS architecture blog; original pp. 43–53	Vendor-specific reference, not a standard
Original lecture	04_API_implementation.pdf (107 pages)	Slides 1–107 adapted page by page
Pets sample	Original lecture screenshots pp. 89–106	Legacy fragments; runtime and repo not supplied

The original Pets example has an exception/handler mismatch (slides 94, 98, 100).